

Spring Core Fundamentals

Complete notes from the Spring Core Fundamentals video.

Focus: Dependency Injection, IoC, IoC Container, Beans, Component Scanning, Reflection, `@Component`, `@Autowired`, Constructor/Setter/Field Injection, `@Primary`, `@Qualifier`, `@Bean`, Configuration, Bean Definitions, and `ApplicationContext`.



Table of Contents

1. [Spring Core Introduction](#)
2. [Project Setup](#)
3. [Revision of Dependency Injection](#)
4. [Problem with Manual Dependency Creation](#)
5. [Dependency Injection](#)
6. [Inversion of Control](#)
7. [IoC Container](#)
8. [Spring Context Dependency](#)
9. [Beans](#)
10. [Annotation-Based Configuration](#)
11. [Java Reflection API](#)
12. [@Component](#)
13. [ApplicationContext](#)
14. [@Configuration](#)
15. [@ComponentScan](#)
16. [Complete Component Scanning Flow](#)
17. [Bean Creation](#)
18. [Dependency Injection by Spring](#)
19. [@Autowired](#)
20. [Constructor Injection](#)
21. [Setter Injection](#)
22. [Field Injection](#)
23. [Comparison of Injection Types](#)
24. [Multiple Beans of Same Type](#)
25. [@Primary](#)
26. [@Qualifier](#)
27. [@Primary vs @Qualifier](#)
28. [@Bean](#)
29. [@Bean Method](#)
30. [@Bean with Interface Implementations](#)
31. [@Bean vs @Component](#)
32. [Bean Naming](#)
33. [Custom Bean Names](#)

34. [@Bean and @Qualifier](#)
 35. [@Bean and @Primary](#)
 36. [@Component + @Bean Together](#)
 37. [Bean Creation Priority](#)
 38. [Why AppConfig is Required](#)
 39. [Configuration vs Main Method](#)
 40. [Annotation-Based Configuration Flow](#)
 41. [XML-Based Configuration](#)
 42. [ApplicationContext Internals](#)
 43. [BeanFactory](#)
 44. [ApplicationContext vs BeanFactory](#)
 45. [Complete Spring Core Flow](#)
 46. [Final Revision](#)
 47. [Interview Questions](#)
-

1. Spring Core Introduction

Spring Framework is built on top of **Spring Core**.

The major concepts covered in this lecture are:

- Dependency Injection
- Inversion of Control
- IoC Container
- Beans
- ApplicationContext
- Component Scanning
- Annotations
- Java Reflection
- Dependency Wiring
- [@Autowired](#)
- [@Primary](#)
- [@Qualifier](#)
- [@Bean](#)

The main objective is to understand:

How Spring creates, manages, and connects objects for us.

2. Project Setup

The video creates a simple Spring Core project.

The project contains classes such as:

```
SpringCoreDemo
|
├─ Main
├─ OrderService
├─ PaymentService
└─ AppConfig
```

The example uses a simple e-commerce scenario.

```
OrderService
  |
  ↓
PaymentService
```

Before placing an order, payment needs to happen.

3. Revision of Dependency Injection

Suppose we have:

```
public class PaymentService {

    public void pay() {
        System.out.println("Payment Done");
    }
}
```

And:

```
public class OrderService {

    private PaymentService paymentService;

    public void placeOrder() {

        paymentService.pay();

        System.out.println("Order Placed");
    }
}
```

```
}  
}
```

`OrderService` depends on `PaymentService`.

Therefore:

```
PaymentService = Dependency  
OrderService   = Dependent
```

4. Problem with Manual Dependency Creation

One approach is:

```
public class OrderService {  
  
    private PaymentService paymentService =  
        new PaymentService();  
}
```

This works, but it creates a problem.

`OrderService` is now responsible for:

1. Handling orders
2. Creating `PaymentService`

This violates the **Single Responsibility Principle**.

The class should focus on orders, not dependency creation.

5. Dependency Injection

Instead of creating the dependency inside the class:

```
new PaymentService();
```

we provide the dependency from outside.

Constructor Injection

```
public class OrderService {  
  
    private PaymentService paymentService;  
  
    public OrderService(PaymentService paymentService) {  
        this.paymentService = paymentService;  
    }  
}
```

Now `OrderService` does not create `PaymentService`.

Someone outside provides it.

For example:

```
PaymentService paymentService =  
    new PaymentService();  
  
OrderService orderService =  
    new OrderService(paymentService);
```

This is:

Dependency Injection

6. Inversion of Control

The previous example still requires us to manually:

- Create `PaymentService`
- Create `OrderService`
- Connect both objects
- Inject the dependency

The video asks:

What if Spring does all of this for us?

This is where **Inversion of Control** comes in.

IoC

IoC = Inversion of Control

Normally:

```
Developer
  ↓
Creates Objects
  ↓
Connects Objects
  ↓
Manages Objects
```

With Spring:

```
Spring
  ↓
Creates Objects
  ↓
Connects Objects
  ↓
Manages Objects
```

The control is transferred from the developer to Spring.

Hence:

Inversion of Control

7. IoC Container

Spring Core provides an **IoC Container**.

The container is responsible for:

- Creating objects
- Managing objects
- Connecting dependencies
- Injecting dependencies
- Managing Beans

Previously our `main()` method was doing:

```
Create PaymentService
      ↓
Create OrderService
      ↓
Pass PaymentService
      ↓
OrderService gets dependency
```

With Spring:

```
IoC Container
  |
  ├── Creates PaymentService
  |
  ├── Creates OrderService
  |
  └── Injects PaymentService
      into OrderService
```

8. Spring Context Dependency

For Spring Core functionality, the video adds the:

```
spring-context
```

dependency.

The Spring Context provides the functionality required to create and manage the application context.

The video also discusses version compatibility with the Spring/Spring Boot version being used.

9. Beans

A very important Spring term is:

Bean

A Bean is an object that is **created and managed by the Spring IoC Container**.

For example:

```
PaymentService Object
    ↓
Managed by Spring
    ↓
PaymentService Bean
```

Similarly:

```
OrderService Object
    ↓
Managed by Spring
    ↓
OrderService Bean
```

Object vs Bean

```
Object
    ↓
Any Java object

Bean
    ↓
Object managed by Spring
```

Therefore:

Every Bean is an Object, but every Object is not necessarily a Bean.

10. Annotation-Based Configuration

Spring can be configured in different ways.

The video focuses on:

Annotation-Based Configuration

Important annotations include:

```
@Component  
@Configuration  
@ComponentScan  
@Autowired  
@Primary  
@Qualifier  
@Bean
```

11. Java Reflection API

Spring needs to inspect classes at runtime.

Java provides:

Reflection API

Reflection allows a program to inspect class information at runtime.

For example:

```
Class<Student> c1 = Student.class;
```

Here:

```
Student.class  
  ↓  
Class object  
  ↓  
Metadata about Student
```

Reflection can provide information about:

- Class name
- Fields
- Constructors
- Methods

- Annotations
 - Modifiers
-

Why Spring Uses Reflection

Spring needs to discover information about classes.

For example:

```
@Component
public class PaymentService {
}
```

Spring needs to know:

```
Is this class a component?
What is its class?
What constructors does it have?
What dependencies does it need?
```

Reflection helps Spring inspect this information.

12. @Component

Now the question is:

How do we tell Spring which classes it should manage?

We use:

```
@Component
```

Example:

```
@Component
public class PaymentService {

    public void pay() {
        System.out.println("Payment Done");
    }
}
```

```
}  
}
```

This tells Spring:

Manage this class as a Spring Bean.

@Component as a Marker

Conceptually:

```
@Component  
  ↓  
Spring recognizes the class  
  ↓  
Spring creates its object  
  ↓  
Object becomes a Bean
```

13. ApplicationContext

Spring needs a container through which it can manage Beans.

This is represented by:

```
ApplicationContext
```

`ApplicationContext` is an **interface**.

Therefore:

```
new ApplicationContext();
```

is not possible.

An implementation must be used.

For annotation configuration:

```
AnnotationConfigApplicationContext
```

is used.

Example:

```
ApplicationContext context =  
    new AnnotationConfigApplicationContext(AppConfig.class);
```

14. @Configuration

Spring needs configuration information.

We create a configuration class:

```
@Configuration  
public class AppConfig {  
  
}
```

`@Configuration` tells Spring:

This class contains Spring configuration.

Example:

```
@Configuration  
public class AppConfig {  
  
}
```

The name `AppConfig` is not mandatory.

You could use:

```
SpringConfig  
ApplicationConfig  
MyConfig
```

The important thing is:

```
@Configuration
```

15. @ComponentScan

Spring needs to know:

Where should I search for `@Component` classes?

For this:

```
@ComponentScan
```

is used.

Example:

```
@Configuration
@ComponentScan("in.coderarmy")
public class AppConfig {

}
```

This tells Spring to scan the specified package.

16. Complete Component Scanning Flow

Suppose we have:

```
in.coderarmy
|
├─ AppConfig
├─ OrderService
└─ PaymentService
```

And:

```
@Component
public class PaymentService {
}
```

```
@Component
public class OrderService {
}
```

Configuration:

```
@Configuration
@ComponentScan("in.coderarmy")
public class AppConfig {
}
```

Spring performs:

```
ApplicationContext starts
↓
Reads AppConfig
↓
Reads @ComponentScan
↓
Scans specified package
↓
Finds @Component classes
↓
OrderService
PaymentService
↓
Creates Bean Definitions
↓
Creates Beans
```

17. Bean Creation

Once Spring discovers:

```
@Component
public class PaymentService {
}
```

it knows that the class needs to be managed.

Conceptually:

```
@Component Class
    ↓
Spring detects it
    ↓
Bean Definition
    ↓
Bean Object
    ↓
Spring manages it
```

18. Dependency Injection by Spring

Suppose:

```
@Component
public class OrderService {

    private PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Spring needs to create:

```
OrderService
```

But `OrderService` requires:

```
PaymentService
```

Spring therefore:

1. Creates PaymentService Bean
2. Creates OrderService
3. Passes PaymentService into OrderService
4. Dependency becomes injected

19. @Autowired

`@Autowired` tells Spring:

Inject the required dependency here.

Example:

```
@Autowired
private PaymentService paymentService;
```

Spring looks for a suitable Bean and injects it.

20. Constructor Injection

Constructor injection means the dependency is provided through the constructor.

Example:

```
@Component
public class OrderService {

    private PaymentService paymentService;

    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

Important Point

With a **single constructor**, Spring can perform constructor injection without explicitly writing:

```
@Autowired
```

So this:

```
public OrderService(PaymentService paymentService) {  
    this.paymentService = paymentService;  
}
```

is sufficient.

Conceptually:

```
Constructor  
  ↓  
Spring sees dependency  
  ↓  
Finds PaymentService Bean  
  ↓  
Passes it to constructor
```

21. Setter Injection

Dependency can also be injected through a setter method.

Example:

```
@Component  
public class OrderService {  
  
    private PaymentService paymentService;  
  
    @Autowired  
    public void setPaymentService(  
        PaymentService paymentService) {
```

```
        this.paymentService = paymentService;
    }
}
```

The sequence is:

```
Create OrderService
      ↓
Create PaymentService
      ↓
Call setter
      ↓
Inject PaymentService
```

The video explains that setter injection happens after the object has been created.

22. Field Injection

Dependency can also be injected directly into a field.

Example:

```
@Component
public class OrderService {

    @Autowired
    private PaymentService paymentService;
}
```

Spring directly injects the dependency into the field.

Conceptually:

```
Create OrderService
      ↓
Find PaymentService
      ↓
Inject into field
```

23. Comparison of Injection Types

Type	Example	Main Idea
Constructor Injection	Constructor parameter	Dependency provided during object creation
Setter Injection	Setter method	Dependency provided through setter
Field Injection	Field	Dependency injected directly into field

Constructor Injection

```
public OrderService(PaymentService paymentService) {  
    this.paymentService = paymentService;  
}
```

Setter Injection

```
@Autowired  
public void setPaymentService(  
    PaymentService paymentService) {  
  
    this.paymentService = paymentService;  
}
```

Field Injection

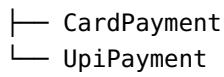
```
@Autowired  
private PaymentService paymentService;
```

24. Multiple Beans of Same Type

Now consider an important situation.

Suppose:

```
Payment  
|
```



Both implement:

```
public interface Payment {  
  
    void pay();  
}
```

CardPayment

```
@Component  
public class CardPayment implements Payment {  
  
    @Override  
    public void pay() {  
        System.out.println("Paying via Card");  
    }  
}
```

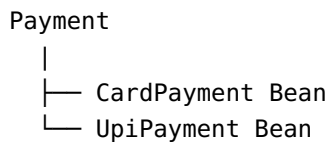
UpiPayment

```
@Component  
public class UpiPayment implements Payment {  
  
    @Override  
    public void pay() {  
        System.out.println("Paying via UPI");  
    }  
}
```

Now:

```
public OrderService(Payment payment) {  
    this.payment = payment;  
}
```

Spring sees:



Spring asks:

Which Payment Bean should I inject?

This creates ambiguity.

25. @Primary

@Primary tells Spring:

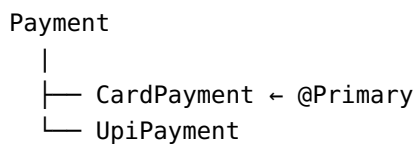
Prefer this Bean when multiple Beans of the same type exist.

Example:

```
@Component
@Primary
public class CardPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Paying via Card");
    }
}
```

Now:



If Spring needs a `Payment` and no specific Bean is selected:

CardPayment

will be selected.

Important

`@Primary` does not remove the other Bean.

Both still exist:

```
CardPayment Bean
UpiPayment Bean
```

But one is preferred.

26. @Qualifier

Instead of defining a default Bean, we can explicitly specify which Bean should be used.

Use:

```
@Qualifier
```

Example:

```
public OrderService(
    @Qualifier("upiPayment") Payment payment) {

    this.payment = payment;
}
```

Now Spring specifically selects:

```
UpiPayment
```

27. @Primary vs @Qualifier

@Primary	@Qualifier
Preferred/default Bean	Specific Bean
Used on Bean	Used at injection point
Spring automatically prefers it	Developer explicitly selects
Good for default implementation	Good when exact implementation is required

Easy way to remember

```
@Primary
↓
"Prefer this"

@Qualifier
↓
"Use this exact one"
```

28. @Bean

So far we used:

```
@Component
```

to tell Spring:

Manage this class.

But Spring also provides another way:

```
@Bean
```

`@Bean` is generally used inside a configuration class.

Example:

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

Spring receives the returned object and manages it as a Bean.

29. @Bean Method

Consider:

```
@Bean
public PaymentService paymentService() {

    return new PaymentService();
}
```

The method:

```
paymentService()
```

creates the object.

Spring manages the returned object.

Flow:

```
@Bean method
    ↓
Method executes
    ↓
new PaymentService()
    ↓
Returned object
```

↓
Spring manages it as Bean

30. @Bean with Interface Implementations

This becomes especially useful with interfaces.

Suppose:

```
public interface Payment {  
  
    void pay();  
}
```

And:

```
public class CardPayment implements Payment {  
  
    @Override  
    public void pay() {  
        System.out.println("Paying via Card");  
    }  
}
```

We can configure:

```
@Configuration  
public class AppConfig {  
  
    @Bean  
    public Payment createCardPayment() {  
        return new CardPayment();  
    }  
}
```

Now Spring manages the returned `Payment` object.

31. @Bean vs @Component

@Component

```
@Component
public class PaymentService {
}
```

Spring discovers the class through component scanning.

@Bean

```
@Bean
public PaymentService paymentService() {
    return new PaymentService();
}
```

Spring gets the object from the configuration method.

Simple Difference

```
@Component
    ↓
"Spring, manage this class."

@Bean
    ↓
"Spring, manage the object returned by this method."
```

32. Bean Naming

Bean names become important when using:

```
@Qualifier
```

For a component:

```
@Component
public class CardPayment {
}
```

the default Bean name is generally:

```
cardPayment
```

For a `@Bean` method:

```
@Bean
public Payment createCardPayment() {
    return new CardPayment();
}
```

the Bean name is:

```
createCardPayment
```

The key difference:

```
@Component
    ↓
Bean name generally based on class name

@Bean
    ↓
Bean name generally based on method name
```

33. Custom Bean Names

Bean names can also be customized.

For example:

```
@Component("card")
public class CardPayment {
}
```

Now the Bean name is:

card

Then:

```
@Qualifier("card")
```

can refer to it.

34. @Bean and @Qualifier

Suppose:

```
@Bean
public Payment createCardPayment() {
    return new CardPayment();
}
```

and:

```
@Bean
public Payment createUpiPayment() {
    return new UpiPayment();
}
```

There are now two Payment Beans:

createCardPayment
createUpiPayment

We can select one:

```

public OrderService(
    @Qualifier("createUpiPayment")
    Payment payment) {

    this.payment = payment;
}

```

Now Spring selects:

```

createUpiPayment
    ↓
UpiPayment

```

35. @Bean and @Primary

We can also use:

```
@Primary
```

with a Bean definition.

Example:

```

@Bean
@Primary
public Payment createCardPayment() {
    return new CardPayment();
}

```

Now if multiple `Payment` Beans exist:

```

createCardPayment ← @Primary
createUpiPayment

```

Spring prefers:

```
createCardPayment
```

36. @Component + @Bean Together

An important question:

What happens if the same class has `@Component` and we also create its object using `@Bean` ?

For example:

```
@Component
public class CardPayment implements Payment {
}
```

and:

```
@Configuration
public class AppConfig {

    @Bean
    public Payment createCardPayment() {
        return new CardPayment();
    }
}
```

At first glance, it may appear that Spring will create two Beans.

But the video explains that the configuration through `@Bean` gets priority for that object in this situation.

Conceptually:

```
@Component
  ↓
Spring can create object

@Bean
  ↓
Configuration explicitly provides object

  ↓
```

@Bean configuration takes priority

The component can therefore be ignored for that same class/object configuration rather than producing two identical Beans.

37. Bean Creation Priority

The important concept from the video is:

Explicit Bean configuration has priority over automatic component creation when both target the same class.

Conceptually:

```
@Component
  ↓
Spring can create object

@Bean
  ↓
Developer explicitly provides object

@Bean
  ↓
Higher priority
```

Therefore, using both does not automatically mean:

2 Beans

for the same configuration target.

38. Why AppConfig is Required

A natural question:

Why do we need a separate AppConfig class?

We could technically put configuration directly inside main().

But that makes the entry point complicated.

Without configuration separation:

```
main()
├─ Create context
├─ Configure components
├─ Configure Beans
├─ Manage dependencies
└─ Start application
```

The `main()` method becomes responsible for too many things.

39. Configuration vs Main Method

The purpose of the configuration class is to separate:

Application Logic

from:

Spring Configuration

Therefore:

```
Main
↓
Start Application
```

while:

```
AppConfig
↓
Spring Configuration
```

Example:

```
@Configuration
@ComponentScan("in.coderarmy")
public class AppConfig {
}
```

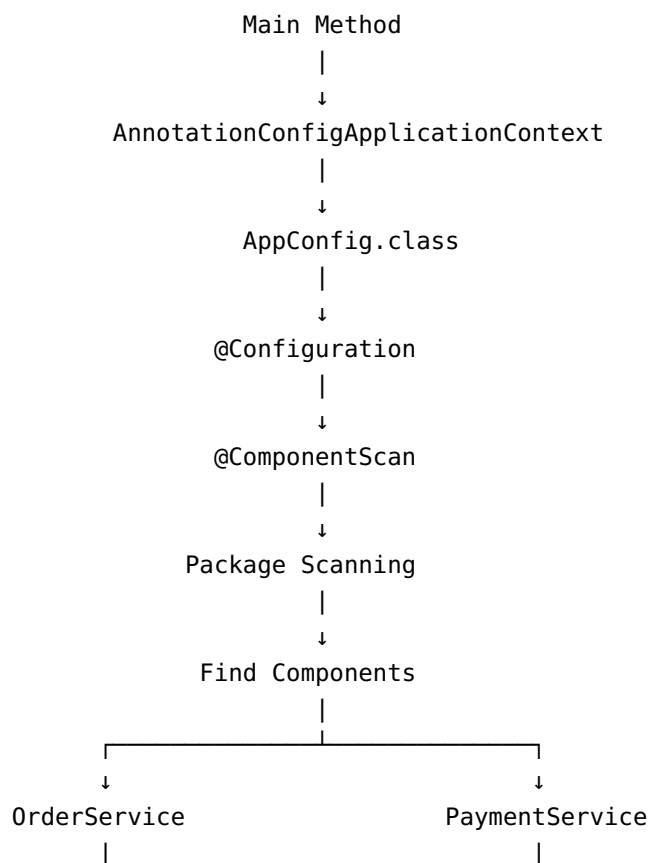
Then:

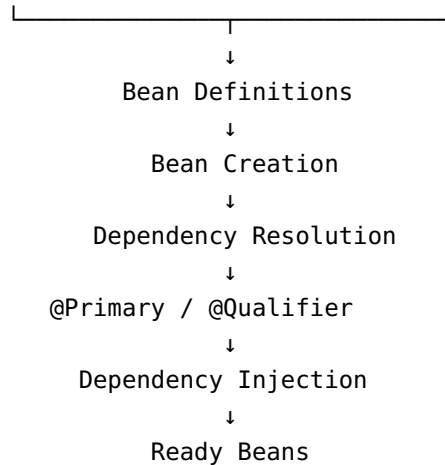
```
ApplicationContext context =
    new AnnotationConfigApplicationContext(
        AppConfig.class
    );
```

This keeps the main method clean.

40. Annotation-Based Configuration Flow

The complete flow is:





41. XML-Based Configuration

The video briefly introduces the older configuration approach:

XML-Based Configuration

Instead of annotations such as:

```
@Component
@Configuration
@ComponentScan
```

Spring can use an XML configuration file.

This approach was commonly used in older Spring applications.

The video points out that XML configuration can become lengthy because it requires many XML tags and configuration entries.

Modern vs Older Approach

```
Annotation Configuration
  ↓
@Component
@Configuration
```

```
@ComponentScan  
@Bean
```

vs.

```
XML Configuration  
  ↓  
XML file  
  ↓  
Beans/configuration declared using XML
```

42. ApplicationContext Internals

The video ends by explaining the internal relationship of:

```
ApplicationContext
```

and:

```
BeanFactory
```

`ApplicationContext` is an interface.

It represents the Spring application container.

43. BeanFactory

`BeanFactory` is also an interface.

Historically, Spring used:

```
BeanFactory
```

as the basic interface for interacting with the IoC container.

The video explains that:

BeanFactory

is a lower-level/basic container abstraction.

44. ApplicationContext vs BeanFactory

The hierarchy can be thought of as:

```
ApplicationContext
  |
  ↓
extends / builds on BeanFactory functionality
  |
  ↓
BeanFactory
```

More specifically, `ApplicationContext` inherits BeanFactory-related capabilities through Spring's interface hierarchy.

Therefore, many BeanFactory operations are available through ApplicationContext.

Why ApplicationContext?

Spring applications generally use:

ApplicationContext

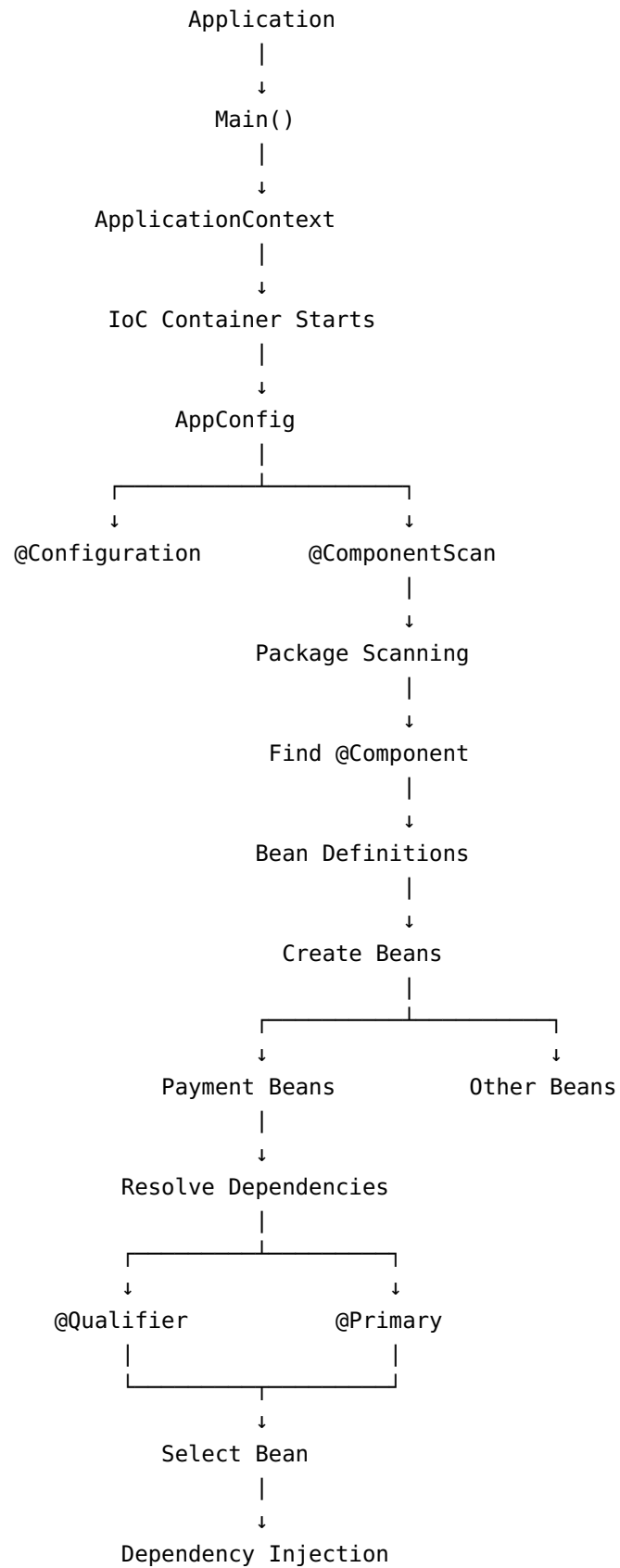
rather than directly interacting with:

BeanFactory

because `ApplicationContext` provides a richer application-level container abstraction.

45. Complete Spring Core Flow

This is the most important flow to remember from the entire video:



|
↓
Managed Beans

46. Final Revision

Dependency Injection

Providing an object's dependency from outside instead of allowing the object to create it itself.

Dependency
↓
Provided externally

Inversion of Control

Transferring control of object creation and management from the developer to Spring.

Developer → Spring

IoC Container

Responsible for:

Create
Manage
Wire
Inject

Spring Beans.

Bean

Object managed by Spring

@Component

Marks a class as a Spring component.

@Configuration

Marks a class as Spring configuration.

@ComponentScan

Tells Spring where to find components.

ApplicationContext

Spring application container interface.

@Autowired

Requests dependency injection.

Constructor Injection

Dependency → Constructor

Setter Injection

Dependency → Setter

Field Injection

Dependency → Field

@Primary

Multiple Beans
↓
Prefer this Bean

@Qualifier

Multiple Beans
↓
Use this specific Bean

@Bean

Method returns object
↓
Spring manages returned object

Reflection

Runtime inspection of class metadata.

Bean Definition

Metadata/information Spring uses to create and manage a Bean.

BeanFactory

Basic IoC container interface.

ApplicationContext

Higher-level Spring application container that builds on BeanFactory functionality.

47. Interview Questions

Q1. What is Dependency Injection?

Dependency Injection is a design technique where a class receives its dependencies from an external source rather than creating them itself.

Q2. What is IoC?

IoC means the control of object creation and management is transferred from the developer to the Spring Framework.

Q3. What is an IoC Container?

It is the Spring container responsible for creating, configuring, wiring, and managing Beans.

Q4. What is a Spring Bean?

A Spring Bean is an object that is created and managed by the Spring IoC Container.

Q5. What does `@Component` do?

It marks a class as a Spring component so that it can be detected during component scanning and managed as a Bean.

Q6. What does `@ComponentScan` do?

It tells Spring which package(s) should be scanned for components.

Q7. What does `@Configuration` do?

It marks a class as a source of Spring configuration.

Q8. What is `ApplicationContext`?

`ApplicationContext` is a Spring interface representing the application-level IoC container.

Q9. What is `AnnotationConfigApplicationContext`?

It is an implementation of `ApplicationContext` used for annotation-based configuration.

Q10. What is `@Autowired`?

It tells Spring to resolve and inject a suitable dependency.

Q11. What happens if multiple Beans of the same type exist?

Spring needs additional information to select one.

We can use:

```
@Primary
```

or:

```
@Qualifier
```

Q12. What is `@Primary`?

It marks one Bean as the preferred Bean when multiple candidates of the same type exist.

Q13. What is `@Qualifier`?

It explicitly specifies which Bean should be injected.

Q14. Difference between `@Primary` and `@Qualifier`?

```
@Primary  
→ Default/preferred Bean  
  
@Qualifier  
→ Specific Bean
```

Q15. What is `@Bean`?

`@Bean` tells Spring to manage the object returned by a configuration method.

Q16. Difference between `@Component` and `@Bean`?

```
@Component  
→ Put annotation on class.  
  
@Bean  
→ Put annotation on configuration method.
```

Q17. What is Reflection?

Reflection allows Java programs to inspect and interact with class metadata at runtime.

Q18. What is a Bean Definition?

A Bean Definition is metadata that Spring uses to understand how a Bean should be created and managed.

Q19. Why do we use a Configuration class?

To keep Spring configuration separate from the application's main entry point and business logic.

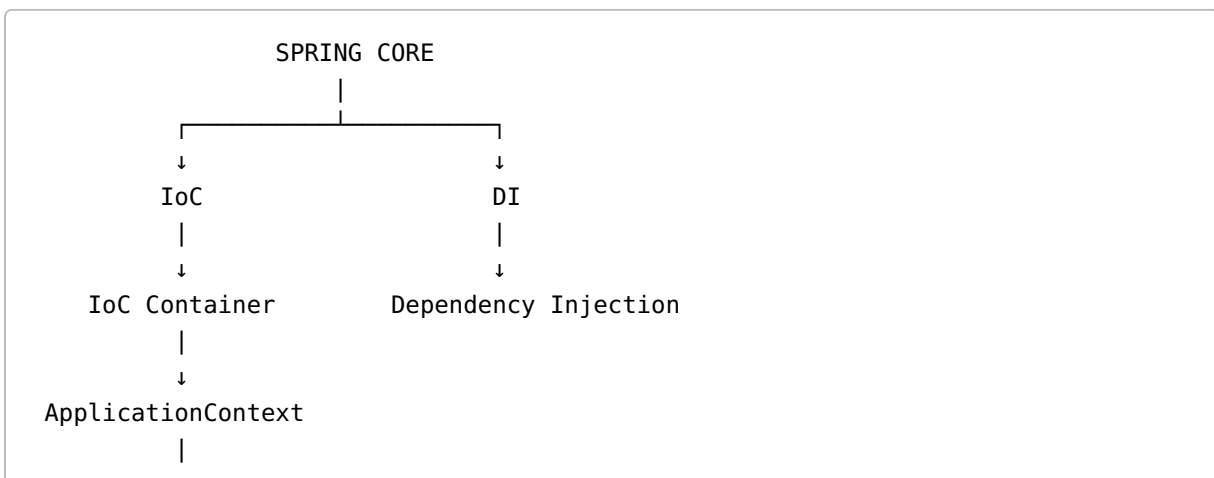
Q20. What is BeanFactory?

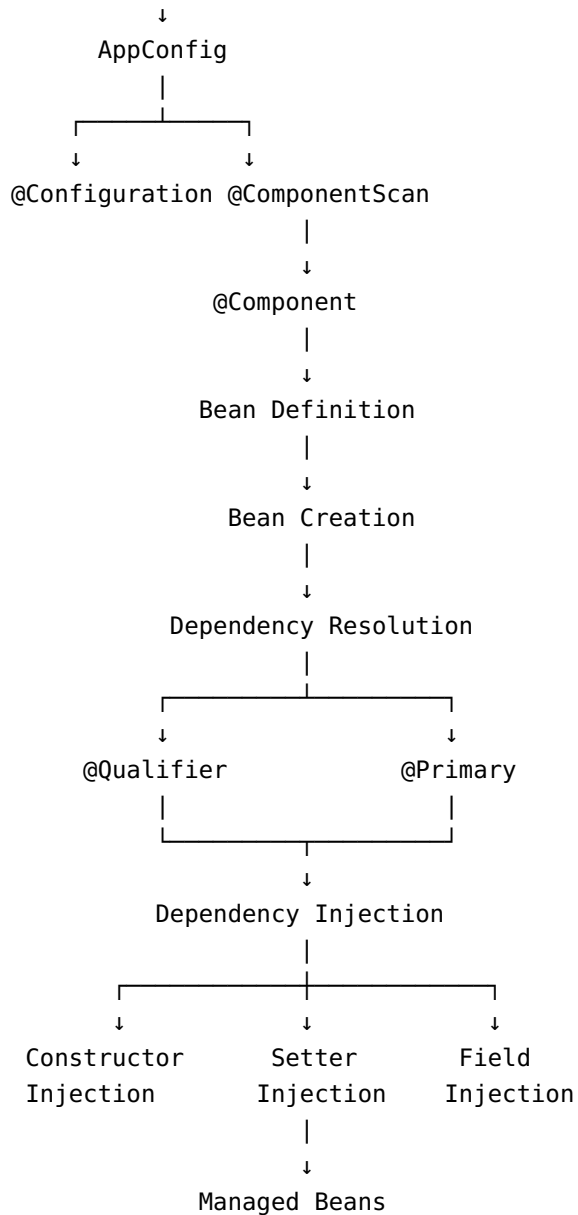
`BeanFactory` is a Spring IoC container interface providing fundamental Bean-management functionality.

Q21. What is the relationship between BeanFactory and ApplicationContext?

`ApplicationContext` is a higher-level container abstraction that extends/inherits BeanFactory-related functionality.

One-Page Memory Map





★ Most Important Things to Remember

1. IoC

Spring controls object creation and management.

2. DI

Spring provides required dependencies.

3. Bean

Object managed by Spring.

4. `@Component`

Spring should manage this class.

5. `@ComponentScan`

Search these packages for components.

6. `@Configuration`

This class contains Spring configuration.

7. `@Bean`

Manage the object returned by this method.

8. `@Primary`

Prefer this Bean.

9. `@Qualifier`

Use this exact Bean.

10. ApplicationContext

Interface representing the Spring application container.

11. BeanFactory

Basic IoC container abstraction.

Final Concept

The entire lecture can be reduced to this:

```
Developer writes classes
↓
Marks required classes/configuration
↓
@Component / @Configuration / @Bean
↓
@ComponentScan
↓
Spring finds classes
↓
Creates Bean Definitions
↓
Creates Beans
↓
Finds dependencies
↓
Resolves multiple Beans
↓
@Qualifier / @Primary
↓
Injects dependencies
↓
IoC Container manages everything
↓
Application uses the Beans
```

Core idea: Spring takes responsibility for creating and managing application objects and connecting their dependencies, allowing the developer to focus on business logic rather than manually managing object creation and wiring.